

BÁO CÁO PHÂN TÍCH HỌC MÁY & GOM CỤM DỮ LIỆU CỜ VUA LICHESS

Đề tài: Phân loại kết quả ván đấu cờ vua & Gom cụm không giám sát

Tập dữ liệu: Lichess Standard Rated Games (Database Lichess)

Mục tiêu thực hiện:

1. **Khảo sát dữ liệu:** Đọc, làm sạch dữ liệu, hiển thị kích thước, kiểu dữ liệu, phân bố nhãn, và thống kê Min/Max/Mean các thuộc tính số thực.
2. **Trực quan hóa 2D:** Trích xuất các thuộc tính liên tục, giảm chiều dữ liệu (PCA & t-SNE) hiển thị trên không gian 2D với màu sắc phân biệt theo nhãn + Trực quan hóa sơ đồ kiến trúc với `graphviz`.
3. **Mô hình phân lớp (10-Fold CV):** Huấn luyện và tinh chỉnh 3 mô hình (**Random Forest, AdaBoost, SVM**), đánh giá F-Score qua 10-Fold Cross-Validation, hiển thị bảng so sánh và cấu trúc cây quyết định `graphviz`.
4. **Gom cụm không giám sát:** Loại bỏ nhãn, gom cụm với **K-Means** và **DBSCAN**, đánh giá chất lượng gom cụm dựa trên nhãn thực tế (**ARI, NMI, V-Measure, Purity, Silhouette**).

```
In [1]: import os
import sys
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import graphviz

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.model_selection import StratifiedKFold
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
```

```

from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.tree import export_graphviz, plot_tree
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import (
    f1_score, accuracy_score, precision_score, recall_score,
    adjusted_rand_score, normalized_mutual_info_score,
    v_measure_score, homogeneity_score, completeness_score,
    fowlkes_mallows_score, silhouette_score
)

# Cấu hình hiển thị đồ thị và seed ngẫu nhiên
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["font.size"] = 11
SEED = 42
np.random.seed(SEED)

print("Đã nạp toàn bộ thư viện thành công!")

```

Đã nạp toàn bộ thư viện thành công!

PHẦN 1: ĐỌC DỮ LIỆU, TIỀN XỬ LÝ VÀ HIỂN THỊ CÁC THÔNG SỐ

Chúng ta tiến hành làm sạch dữ liệu:

- Trích xuất các đặc trưng Elo người chơi: `white_rating`, `black_rating`.
- Tính hiệu số trình độ: `rating_diff = white_rating - black_rating`.
- Phân loại tính điểm: `rated` (1: Rated, 0: Casual).
- Tính độ sâu lý thuyết khai cuộc: `opening_ply` (từ tên khai cuộc PGN, tránh data leakage).
- Mã hóa nhãn kết quả `ResultEncoded` :
 - 0 : Đen thắng (0-1)
 - 1 : Hòa (1/2-1/2)
 - 2 : Trắng thắng (1-0)

```

In [2]: def clean_moves(moves_str):
        if not isinstance(moves_str, str):

```

```

        return ""
    moves_str = re.sub(r"\{[^}]*\}", "", moves_str)
    moves_str = re.sub(r"\d+\.\s*", "", moves_str)
    moves_str = re.sub(r"1-0|0-1|1/2-1/2|\*", "", moves_str)
    return " ".join(moves_str.split())

def extract_opening_ply(opening_str):
    if not isinstance(opening_str, str) or opening_str == "?" or not opening_str:
        return 8.0
    match_triple = re.findall(r"(\d+)\.\.\.", opening_str)
    if match_triple:
        return float(int(match_triple[-1]) * 2)
    match_single = re.findall(r"(\d+)\.", opening_str)
    if match_single:
        return float(int(match_single[-1]) * 2 - 1)
    if ":" in opening_str:
        var_part = opening_str.split(":", 1)[1].strip()
        plies = len(clean_moves(var_part).split())
        if plies > 0:
            return float(min(max(plies, 2), 20))
    return 8.0

# 1. Đọc dữ liệu vào bộ nhớ
raw_csv_path = "data/processed_games.csv"
if not os.path.exists(raw_csv_path):
    raise FileNotFoundError(f"Không tìm thấy file {raw_csv_path}")

raw_df = pd.read_csv(raw_csv_path, dtype=str)
df = raw_df.copy()

# Xử lý ratings
df["white_rating"] = pd.to_numeric(df["WhiteElo"], errors="coerce")
df["black_rating"] = pd.to_numeric(df["BlackElo"], errors="coerce")
df = df.dropna(subset=["white_rating", "black_rating"])
df["white_rating"] = df["white_rating"].astype(float)
df["black_rating"] = df["black_rating"].astype(float)

# Lọc khoảng Elo hợp lệ
df = df[(df["white_rating"] >= 600) & (df["white_rating"] <= 3500)]
df = df[(df["black_rating"] >= 600) & (df["black_rating"] <= 3500)]

```

```

# Tính các thuộc tính
df["rating_diff"] = df["white_rating"] - df["black_rating"]
if "Event" in df.columns:
    df["rated"] = df["Event"].astype(str).apply(lambda ev: 1 if "rated" in ev.lower() else 0)
else:
    df["rated"] = 1
df["opening_ply"] = df["Opening"].apply(extract_opening_ply)

# Mã hóa nhãn
result_map = {"0-1": 0, "1/2-1/2": 1, "1-0": 2}
df = df[df["Result"].isin(result_map.keys())].copy()
df["ResultEncoded"] = df["Result"].map(result_map).astype(int)

# Lưu bộ dữ liệu sạch
clean_csv_path = "data/filtered_processed_games.csv"
os.makedirs(os.path.dirname(clean_csv_path), exist_ok=True)
df.to_csv(clean_csv_path, index=False)
print(f"Đã làm sạch và nạp thành công {len(df):,} ván cờ vào bộ nhớ!")

```

Đã làm sạch và nạp thành công 10,000 ván cờ vào bộ nhớ!

```

In [3]: # 1.a. Kích thước và chiều của dữ liệu
print("=" * 60)
print("1.a. KÍCH THƯỚC VÀ CHIỀU CỦA DỮ LIỆU:")
print("=" * 60)
print(f" - Số lượng mẫu dữ liệu (Số hàng - n_samples): {df.shape[0]:,}")
print(f" - Số lượng thuộc tính (Số cột - n_features) : {df.shape[1]:,}")
print(f" - Kích thước DataFrame (Shape) : {df.shape}")

# 1.b. Kiểu dữ liệu của các thuộc tính
print("\n" + "=" * 60)
print("1.b. KIỂU DỮ LIỆU CỦA CÁC THUỘC TÍNH (DTYPES):")
print("=" * 60)
dtypes_table = pd.DataFrame({"Thuộc tính": df.columns, "Kiểu dữ liệu": df.dtypes.astype(str)})
display(dtypes_table)

# 1.c. Số lượng thực thể của các giá trị nhãn
print("\n" + "=" * 60)
print("1.c. SỐ LƯỢNG THỰC THỂ CỦA CÁC GIÁ TRỊ NHÃN:")
print("=" * 60)
label_desc = {0: "0 (Black win - Đen thắng)", 1: "1 (Draw - Hòa)", 2: "2 (White win - Trắng thắng)"}

```

```

class_counts = df["ResultEncoded"].value_counts().sort_index()
for lbl, cnt in class_counts.items():
    pct = cnt / len(df) * 100
    print(f" - Nhân {label_desc[lbl]:<30}: {cnt:>6,} mẫu ({pct:.2f}%)"

# 1.d. Giá trị Min, Max, Mean của các cột thuộc tính số thực / Liên tục
print("\n" + "=" * 60)
print("1.d. THỐNG KÊ MIN, MAX, TRUNG BÌNH CÁC CỘT SỐ LIÊN TỤC:")
print("=" * 60)
continuous_cols = ["white_rating", "black_rating", "rating_diff", "opening_ply"]
stats_summary = []
for c in continuous_cols:
    stats_summary.append({
        "Thuộc tính": c,
        "Min (Bé nhất)": df[c].min(),
        "Max (Lớn nhất)": df[c].max(),
        "Mean (Trung bình)": round(df[c].mean(), 2),
        "Median (Trung vị)": round(df[c].median(), 2),
        "Std Dev (Độ lệch chuẩn)": round(df[c].std(), 2)
    })
stats_df = pd.DataFrame(stats_summary)
display(stats_df)

```

=====

1.a. KÍCH THƯỚC VÀ CHIỀU CỦA DỮ LIỆU:

=====

- Số lượng mẫu dữ liệu (Số hàng - n_samples): 10,000
- Số lượng thuộc tính (Số cột - n_features) : 17
- Kích thước DataFrame (Shape) : (10000, 17)

=====

1.b. KIỂU DỮ LIỆU CỦA CÁC THUỘC TÍNH (DTYPES):

=====

	Thuộc tính	Kiểu dữ liệu
	White	object
	Black	object
	WhiteElo	object
	BlackElo	object
	Result	object
	ECO	object
	Opening	object
	TimeControl	object
	Termination	object
	Moves	object
	Event	object
	white_rating	float64
	black_rating	float64
	rating_diff	float64
	rated	int64
	opening_ply	float64
	ResultEncoded	int64

```
=====
1.c. SỐ LƯỢNG THỰC THỂ CỦA CÁC GIÁ TRỊ NHÃN:
=====
- Nhãn 0 (Black win - Đen thắng)      :  4,757 mẫu (47.57%)
- Nhãn 1 (Draw - Hòa)                  :    332 mẫu (3.32%)
- Nhãn 2 (White win - Trắng thắng)     :  4,911 mẫu (49.11%)

=====
1.d. THỐNG KÊ MIN, MAX, TRUNG BÌNH CÁC CỘT SỐ LIÊN TỤC:
=====
```

	Thuộc tính	Min (Bé nhất)	Max (Lớn nhất)	Mean (Trung bình)	Median (Trung vị)	Std Dev (Độ lệch chuẩn)
0	white_rating	860.0	2685.0	1634.58	1621.0	272.85
1	black_rating	866.0	2698.0	1635.37	1621.0	272.89
2	rating_diff	-1265.0	1153.0	-0.79	-1.0	236.47
3	opening_ply	2.0	15.0	4.77	4.0	2.74

PHẦN 2: TRỰC QUAN HÓA DỮ LIỆU LIÊN TỤC, GIẢM CHIỀU 2D & SƠ ĐỒ GRAPHVIZ

- **Tập thuộc tính liên tục:** ['white_rating', 'black_rating', 'rating_diff', 'opening_ply']
- **Chuẩn hóa dữ liệu:** Sử dụng `StandardScaler` để đưa các thuộc tính về cùng thang đo chuẩn ($\mu = 0, \sigma = 1$).
- **Thu giảm số chiều:**
 - **PCA 2D:** Chiếu dữ liệu lên 2 trục trực giao có phương sai lớn nhất.
 - **t-SNE 2D:** Thu giảm phi tuyến tính bảo toàn cấu trúc lân cận cụm.
- **Màu sắc phân biệt:**
 -  **Đỏ:** 0 (Black win)
 -  **Cam:** 1 (Draw)
 -  **Xanh lá:** 2 (White win)
- **Sơ đồ quy trình (Pipeline Architecture):** Được vẽ và hiển thị trực quan bằng thư viện `graphviz`.

```
In [4]: # Trực quan hóa sơ đồ kiến trúc quy trình học máy với Graphviz
pipeline_dot = graphviz.Digraph('ML_Pipeline', comment='Machine Learning Workflow')
pipeline_dot.attr(rankdir='LR', size='12,5')
pipeline_dot.attr('node', shape='box', style='filled,rounded', fontname='Arial', fontsize='10')

# Khởi tạo các Node trong sơ đồ
pipeline_dot.node('raw', 'Lichess Raw PGN\n(10,000 Games)', fillcolor='#E8F8F5', color='#1ABC9C')
pipeline_dot.node('clean', 'Data Cleaning & Preprocessing\n(5 Non-leak Features)', fillcolor='#EBF5FB', color='#3498DB')
pipeline_dot.node('scale', 'StandardScaler\n(Feature Normalization)', fillcolor='#FEF9E7', color='#F1C40F')

pipeline_dot.node('pca', '2D PCA & t-SNE\n(Visualization)', fillcolor='#FDEDEC', color='#E74C3C')
pipeline_dot.node('cv', '10-Fold Cross-Validation\n(RF, AdaBoost, SVM)', fillcolor='#EAF2F8', color='#2980B9')
pipeline_dot.node('cluster', 'Unsupervised Clustering\n(K-Means & DBSCAN)', fillcolor='#F5EEF8', color='#8E44AD')

# Tạo các cạnh kết nối
pipeline_dot.edge('raw', 'clean')
pipeline_dot.edge('clean', 'scale')
pipeline_dot.edge('scale', 'pca')
pipeline_dot.edge('scale', 'cv')
pipeline_dot.edge('scale', 'cluster')

pipeline_dot
```



```

-----
FileNotFoundError                                Traceback (most recent call last)
File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\backend\execute.py:76, in run_check(cmd, input_lines, encoding, quiet, **kwargs)
    75         kwargs['stdout'] = kwargs['stderr'] = subprocess.PIPE
---> 76     proc = _run_input_lines(cmd, input_lines, kwargs=kwargs)
    77 else:

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\backend\execute.py:96, in _run_input_lines(cmd, input_lines, kwargs)
    95 def _run_input_lines(cmd, input_lines, *, kwargs):
---> 96     popen = subprocess.Popen(cmd, stdin=subprocess.PIPE, **kwargs)
    98     stdin_write = popen.stdin.write

File ~\AppData\Local\Programs\Python\Python313\Lib\subprocess.py:1039, in Popen.__init__(self, args, bufsize, executable, stdin, stdout, stderr, preexec_fn, close_fds, shell, cwd, env, universal_newlines, startupinfo, creationflags, restore_signals, start_new_session, pass_fds, user, group, extra_groups, encoding, errors, text, umask, pipesize, process_group)
    1036         self.stderr = io.TextIOWrapper(self.stderr,
    1037                                         encoding=encoding, errors=errors)
-> 1039     self._execute_child(args, executable, preexec_fn, close_fds,
    1040                          pass_fds, cwd, env,
    1041                          startupinfo, creationflags, shell,
    1042                          p2cread, p2cwrite,
    1043                          c2pread, c2pwrite,
    1044                          errread, errwrite,
    1045                          restore_signals,
    1046                          gid, gids, uid, umask,
    1047                          start_new_session, process_group)
    1048 except:
    1049     # Cleanup if the child failed starting.

File ~\AppData\Local\Programs\Python\Python313\Lib\subprocess.py:1554, in Popen._execute_child(self, args, executable, preexec_fn, close_fds, pass_fds, cwd, env, startupinfo, creationflags, shell, p2cread, p2cwrite, c2pread, c2pwrite, errread, errwrite, unused_restore_signals, unused_gid, unused_gids, unused_uid, unused_umask, unused_start_new_session, unused_process_group)
    1553 try:
-> 1554     hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
    1555                                               # no special security
    1556                                               None, None,
    1557                                               int(not close_fds),
    1558                                               creationflags,

```

```

1559         env,
1560         cwd,
1561         startupinfo)
1562 finally:
1563     # Child is launched. Close the parent's copy of those pipe
1564     # handles that only the child should have open. You need
1565     (...) 1567     # pipe will not close when the child process exits and the
1566     1568     # ReadFile will hang.

```

FileNotFoundError: [WinError 2] The system cannot find the file specified

The above exception was the direct cause of the following exception:

```

ExecutableNotFound                                Traceback (most recent call last)
File ~\AppData\Roaming\Python\Python313\site-packages\IPython\core\formatters.py:1036, in MimeBundleFormatter.__call__(self, obj, include, exclude)
    1033     method = get_real_method(obj, self.print_method)
    1035     if method is not None:
-> 1036         return method(include=include, exclude=exclude)
    1037     return None
    1038 else:

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\jupyter_integration.py:98, in JupyterIntegration._repr_mimebundle(self, include, exclude, **_)
     96 include = set(include) if include is not None else {self._jupyter_mimetype}
     97 include -= set(exclude or [])
--> 98 return {mimetype: getattr(self, method_name)()
     99         for mimetype, method_name in MIME_TYPES.items()
    100         if mimetype in include}

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\jupyter_integration.py:112, in JupyterIntegration._repr_image_svg_xml(self)
    110 def _repr_image_svg_xml(self) -> str:
    111     """Return the rendered graph as SVG string."""
--> 112     return self.pipe(format=, encoding=SVG_ENCODING)

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\piping.py:104, in Pipe.pipe(self, format, renderer, formatter, neato_no_op, quiet, engine, encoding)
     55 def pipe(self,
     56         format: typing.Optional[str] = None,
     57         renderer: typing.Optional[str] = None,

```

```

(...)    61         engine: typing.Optional[str] = None,
          62         encoding: typing.Optional[str] = None) -> typing.Union[bytes, str]:
          63         """Return the source piped through the Graphviz layout command.
          64
          65         Args:
          (...)    102         '<?xml version='
          103         """
--> 104         return self._pipe_legacy(format,
          105                                renderer=renderer,
          106                                formatter=formatter,
          107                                neato_no_op=neato_no_op,
          108                                quiet=quiet,
          109                                engine=engine,
          110                                encoding=encoding)

```

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz_tools.py:185, in `deprecate_positional_args.<locals>.decorator.<locals>.wrapper(*args, **kwargs)`

```

          177         wanted = ', '.join(f'{name}={value!r}'
          178                               for name, value in deprecated.items())
          179         warnings.warn(f'The signature of {func_name} will be reduced'
          180                       f' to {supported_number} positional arg{s_}{qualification}'
          181                       f' {list(supported)}: pass {wanted} as keyword arg{s_}',
          182                       stacklevel=stacklevel,
          183                       category=category)
--> 185         return func(*args, **kwargs)

```

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\piping.py:121, in `Pipe._pipe_legacy(self, format, renderer, formatter, neato_no_op, quiet, engine, encoding)`

```

          112         @_tools.deprecate_positional_args(supported_number=1, ignore_arg='self')
          113         def _pipe_legacy(self,
          114                         format: typing.Optional[str] = None,
          (...)    119                         engine: typing.Optional[str] = None,
          120                         encoding: typing.Optional[str] = None) -> typing.Union[bytes, str]:
--> 121         return self._pipe_future(format,
          122                                renderer=renderer,
          123                                formatter=formatter,
          124                                neato_no_op=neato_no_op,
          125                                quiet=quiet,
          126                                engine=engine,
          127                                encoding=encoding)

```

```

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\piping.py:149, in Pipe._pipe_future(self, format, renderer, formatter, neato_no_op, quiet, engine, encoding)
    146 if encoding is not None:
    147     if codecs.lookup(encoding) is codecs.lookup(self.encoding):
    148         # common case: both stdin and stdout need the same encoding
--> 149     return self._pipe_lines_string(*args, encoding=encoding, **kwargs)
    150     try:
    151         raw = self._pipe_lines(*args, input_encoding=self.encoding, **kwargs)

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\backend\piping.py:212, in pipe_lines_string(engine, format, input_lines, encoding, renderer, formatter, neato_no_op, quiet)
    206 cmd = dot_command.command(engine, format,
    207                             renderer=renderer,
    208                             formatter=formatter,
    209                             neato_no_op=neato_no_op)
    210 kwargs = {'input_lines': input_lines, 'encoding': encoding}
--> 212 proc = execute.run_check(cmd, capture_output=True, quiet=quiet, **kwargs)
    213 return proc.stdout

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\backend\execute.py:81, in run_check(cmd, input_lines, encoding, quiet, **kwargs)
    79 except OSError as e:
    80     if e.errno == errno.ENOENT:
---> 81         raise ExecutableNotFound(cmd) from e
    82     raise
    84 if not quiet and proc.stderr:

ExecutableNotFound: failed to execute WindowsPath('dot'), make sure the Graphviz executables are on your systems' PATH

```

Out[4]: <graphviz.graphs.Digraph at 0x25800f18d70>

```

In [5]: # Trích xuất dữ liệu liên tục và chuẩn hóa
continuous_cols = ["white_rating", "black_rating", "rating_diff", "opening_ply"]
X_cont = df[continuous_cols].copy()
y = df["ResultEncoded"].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_cont)

# 1. PCA 2D
pca = PCA(n_components=2, random_state=SEED)

```

```

X_pca = pca.fit_transform(X_scaled)
var_ratio = pca.explained_variance_ratio_

# 2. t-SNE 2D (Sample 3,000 điểm để hiển thị rõ ràng và tối ưu thời gian)
sample_size = min(3000, len(df))
sample_indices = np.random.choice(len(df), size=sample_size, replace=False)
tsne = TSNE(n_components=2, perplexity=30, random_state=SEED, max_iter=1000)
X_tsne_sample = tsne.fit_transform(X_scaled[sample_indices])
y_sample = y[sample_indices]

# Vẽ biểu đồ 2D
fig, axes = plt.subplots(1, 2, figsize=(18, 7))
palette = {0: '#e74c3c', 1: '#f39c12', 2: '#2ecc71'}
label_names = {0: "0 (Black win)", 1: "1 (Draw)", 2: "2 (White win)"}

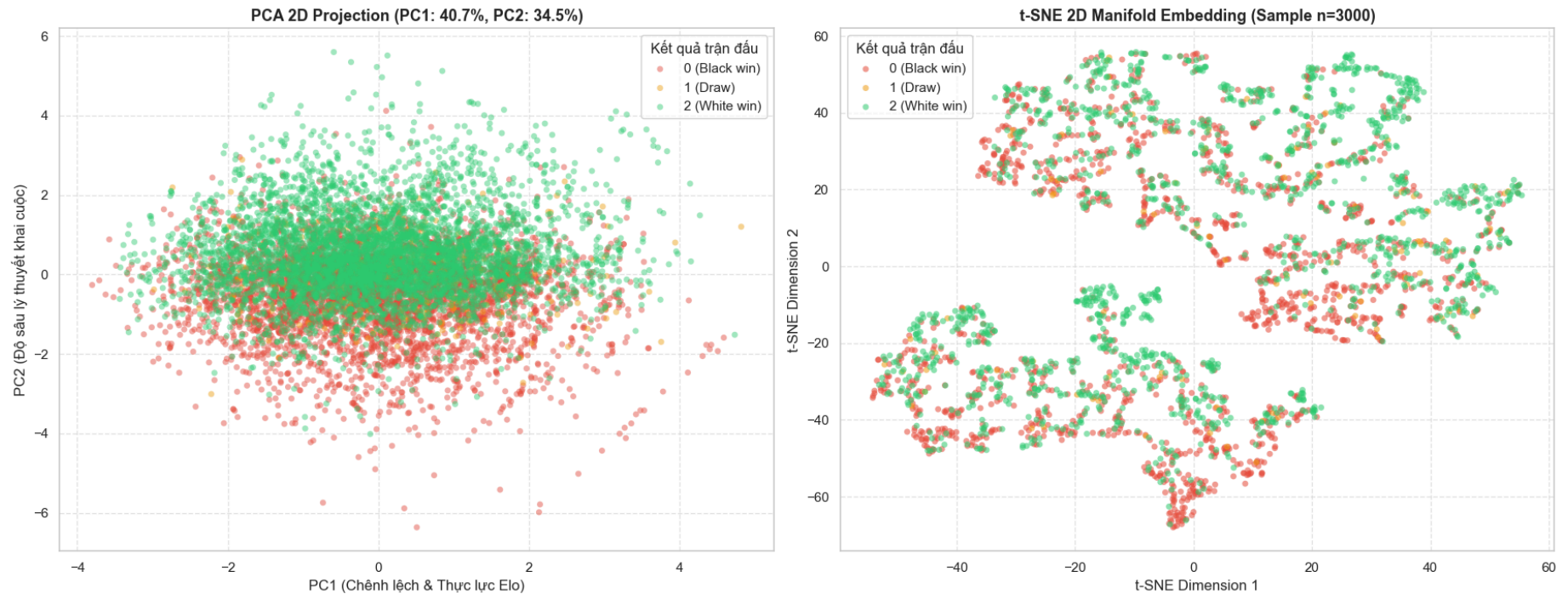
# Subplot 1: PCA 2D
for lbl, color in palette.items():
    mask = (y == lbl)
    axes[0].scatter(
        X_pca[mask, 0], X_pca[mask, 1],
        c=color, label=label_names[lbl],
        alpha=0.45, s=25, edgecolors='none'
    )
axes[0].set_title(f"PCA 2D Projection (PC1: {var_ratio[0]*100:.1f}%, PC2: {var_ratio[1]*100:.1f}%)", fontsize=13, fontweight='bold')
axes[0].set_xlabel("PC1 (Chênh lệch & Thực lực Elo)")
axes[0].set_ylabel("PC2 (Độ sâu lý thuyết khai cuộc)")
axes[0].grid(True, linestyle="--", alpha=0.5)
axes[0].legend(title="Kết quả trận đấu", loc="best")

# Subplot 2: t-SNE 2D
for lbl, color in palette.items():
    mask = (y_sample == lbl)
    axes[1].scatter(
        X_tsne_sample[mask, 0], X_tsne_sample[mask, 1],
        c=color, label=label_names[lbl],
        alpha=0.55, s=25, edgecolors='none'
    )
axes[1].set_title(f"t-SNE 2D Manifold Embedding (Sample n={sample_size})", fontsize=13, fontweight='bold')
axes[1].set_xlabel("t-SNE Dimension 1")
axes[1].set_ylabel("t-SNE Dimension 2")
axes[1].grid(True, linestyle="--", alpha=0.5)

```

```
axes[1].legend(title="Kết quả trận đấu", loc="best")

plt.tight_layout()
plt.show()
```



PHẦN 3: HUẤN LUYỆN VÀ ĐÁNH GIÁ 10-FOLD CROSS-VALIDATION

- **5 Đặc trưng đầu vào:** ['white_rating', 'black_rating', 'rating_diff', 'rated', 'opening_ply']
- **Phương pháp kiểm thử:** $k = 10$ Fold Stratified Cross-Validation
- **3 Mô hình phân lớp đã tinh chỉnh tham số:**
 1. **Random Forest:** `n_estimators=120`, `max_depth=12`, `min_samples_split=6`, `class_weight='balanced'`
 2. **AdaBoost:** `n_estimators=100`, `learning_rate=0.15`
 3. **SVM (RBF Kernel):** `SVC(C=1.5, kernel='rbf', gamma='scale', class_weight='balanced')` kết hợp `StandardScaler` trong `Pipeline`.

```

In [6]: feature_cols = ["white_rating", "black_rating", "rating_diff", "rated", "opening_ply"]
X = df[feature_cols].copy()
y = df["ResultEncoded"].values

models = {
    "Random Forest": RandomForestClassifier(
        n_estimators=120,
        max_depth=12,
        min_samples_split=6,
        min_samples_leaf=3,
        class_weight="balanced",
        random_state=SEED,
        n_jobs=-1
    ),
    "AdaBoost": AdaBoostClassifier(
        n_estimators=100,
        learning_rate=0.15,
        random_state=SEED
    ),
    "SVM (RBF Kernel)": Pipeline([
        ("scaler", StandardScaler()),
        ("svm", SVC(
            C=1.5,
            kernel="rbf",
            gamma="scale",
            class_weight="balanced",
            random_state=SEED
        ))
    ])
}

kfold = StratifiedKFold(n_splits=10, shuffle=True, random_state=SEED)
cv_results = []

for model_name, model in models.items():
    print(f"[*] Đang chạy 10-Fold CV cho mô hình: {model_name}...")
    fold_macro_f1, fold_weighted_f1, fold_acc = [], [], []
    f1_black, f1_draw, f1_white = [], [], []

    for train_idx, val_idx in kfold.split(X, y):

```

```

X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
y_train, y_val = y[train_idx], y[val_idx]

model.fit(X_train, y_train)
y_pred = model.predict(X_val)

fold_macro_f1.append(f1_score(y_val, y_pred, average="macro", zero_division=0))
fold_weighted_f1.append(f1_score(y_val, y_pred, average="weighted", zero_division=0))
fold_acc.append(accuracy_score(y_val, y_pred))

per_f1 = f1_score(y_val, y_pred, average=None, zero_division=0)
f1_black.append(per_f1[0])
f1_draw.append(per_f1[1] if len(per_f1) > 1 else 0)
f1_white.append(per_f1[2] if len(per_f1) > 2 else 0)

cv_results.append({
    "Mô Hình": model_name,
    "Macro F1-Score (Mean ± Std)": f"{np.mean(fold_macro_f1)*100:.2f}% ± {np.std(fold_macro_f1)*100:.2f}%",
    "Weighted F1-Score": f"{np.mean(fold_weighted_f1)*100:.2f}%",
    "Accuracy": f"{np.mean(fold_acc)*100:.2f}%",
    "F1 (Black win)": f"{np.mean(f1_black)*100:.2f}%",
    "F1 (Draw)": f"{np.mean(f1_draw)*100:.2f}%",
    "F1 (White win)": f"{np.mean(f1_white)*100:.2f}%",
    "macro_f1_num": np.mean(fold_macro_f1) * 100,
    "weighted_f1_num": np.mean(fold_weighted_f1) * 100,
    "acc_num": np.mean(fold_acc) * 100
})

comparison_df = pd.DataFrame(cv_results)
print("\n" + "=" * 80)
print("BẢNG SO SÁNH HIỆU NĂNG CÁC MÔ HÌNH VỚI ĐỘ ĐO F-SCORE (10-FOLD CV)")
print("=" * 80)
display(comparison_df[[
    "Mô Hình", "Macro F1-Score (Mean ± Std)", "Weighted F1-Score", "Accuracy",
    "F1 (Black win)", "F1 (Draw)", "F1 (White win)"
]])

[*] Đang chạy 10-Fold CV cho mô hình: Random Forest...
[*] Đang chạy 10-Fold CV cho mô hình: AdaBoost...
[*] Đang chạy 10-Fold CV cho mô hình: SVM (RBF Kernel)...
```


=====

BẢNG SO SÁNH HIỆU NĂNG CÁC MÔ HÌNH VỚI ĐỘ ĐO F-SCORE (10-FOLD CV)

=====

	Mô Hình	Macro F1-Score (Mean ± Std)	Weighted F1-Score	Accuracy	F1 (Black win)	F1 (Draw)	F1 (White win)
0	Random Forest	44.22% ± 1.65%	62.53%	62.88%	64.69%	3.56%	64.42%
1	AdaBoost	43.96% ± 0.62%	63.74%	64.86%	66.12%	0.00%	65.75%
2	SVM (RBF Kernel)	40.74% ± 0.52%	55.57%	49.48%	57.75%	7.78%	56.69%

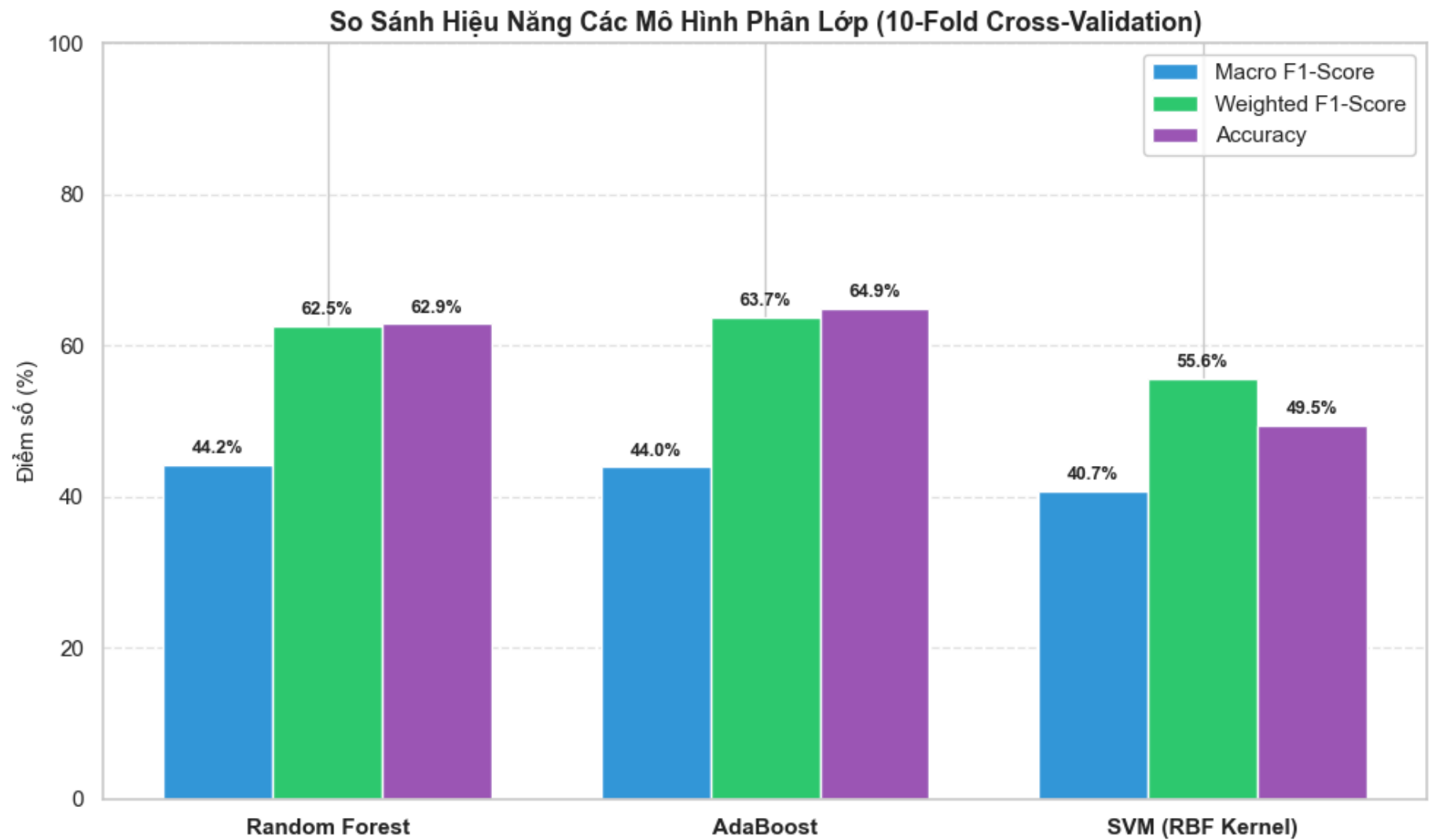
```
In [7]: # Vẽ biểu đồ so sánh hiệu năng các mô hình
plt.figure(figsize=(10, 6))
x_pos = np.arange(len(cv_results))
macro_scores = [r["macro_f1_num"] for r in cv_results]
weighted_scores = [r["weighted_f1_num"] for r in cv_results]
acc_scores = [r["acc_num"] for r in cv_results]

width = 0.25
plt.bar(x_pos - width, macro_scores, width=width, label="Macro F1-Score", color="#3498db")
plt.bar(x_pos, weighted_scores, width=width, label="Weighted F1-Score", color="#2ecc71")
plt.bar(x_pos + width, acc_scores, width=width, label="Accuracy", color="#9b59b6")

plt.xticks(x_pos, [r["Mô Hình"] for r in cv_results], fontsize=11, fontweight="bold")
plt.ylabel("Điểm số (%)", fontsize=11)
plt.title("So Sánh Hiệu Năng Các Mô Hình Phân Lớp (10-Fold Cross-Validation)", fontsize=13, fontweight="bold")
plt.ylim(0, 100)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.legend(loc="upper right", framealpha=0.9)

for i in range(len(cv_results)):
    plt.text(i - width, macro_scores[i] + 1.5, f"{macro_scores[i]:.1f}%", ha='center', fontsize=9, fontweight="bold")
    plt.text(i, weighted_scores[i] + 1.5, f"{weighted_scores[i]:.1f}%", ha='center', fontsize=9, fontweight="bold")
    plt.text(i + width, acc_scores[i] + 1.5, f"{acc_scores[i]:.1f}%", ha='center', fontsize=9, fontweight="bold")

plt.tight_layout()
plt.show()
```



```
In [8]: # Trục quan hóa Cây Quyết định (Decision Tree) từ Random Forest bằng Graphviz
rf_model = models["Random Forest"]
rf_model.fit(X, y)
rf_single_tree = rf_model.estimators_[0]

dot_tree_data = export_graphviz(
    rf_single_tree,
```

```
max_depth=3,  
feature_names=feature_cols,  
class_names=["Black win", "Draw", "White win"],  
filled=True,  
rounded=True,  
special_characters=True  
)  
  
tree_graph = graphviz.Source(dot_tree_data)  
tree_graph
```

```

-----
FileNotFoundError                                Traceback (most recent call last)
File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\backend\execute.py:76, in run_check(cmd, input_lines,
encoding, quiet, **kwargs)
    75         kwargs['stdout'] = kwargs['stderr'] = subprocess.PIPE
---> 76     proc = _run_input_lines(cmd, input_lines, kwargs=kwargs)
    77 else:

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\backend\execute.py:96, in _run_input_lines(cmd, input
_lines, kwargs)
    95 def _run_input_lines(cmd, input_lines, *, kwargs):
---> 96     popen = subprocess.Popen(cmd, stdin=subprocess.PIPE, **kwargs)
    98     stdin_write = popen.stdin.write

File ~\AppData\Local\Programs\Python\Python313\Lib\subprocess.py:1039, in Popen.__init__(self, args, bufsize, executable, stdi
n, stdout, stderr, preexec_fn, close_fds, shell, cwd, env, universal_newlines, startupinfo, creationflags, restore_signals, sta
rt_new_session, pass_fds, user, group, extra_groups, encoding, errors, text, umask, pipesize, process_group)
   1036         self.stderr = io.TextIOWrapper(self.stderr,
   1037                                         encoding=encoding, errors=errors)
-> 1039     self._execute_child(args, executable, preexec_fn, close_fds,
   1040                          pass_fds, cwd, env,
   1041                          startupinfo, creationflags, shell,
   1042                          p2cread, p2cwrite,
   1043                          c2pread, c2pwrite,
   1044                          errread, errwrite,
   1045                          restore_signals,
   1046                          gid, gids, uid, umask,
   1047                          start_new_session, process_group)
   1048 except:
   1049     # Cleanup if the child failed starting.

File ~\AppData\Local\Programs\Python\Python313\Lib\subprocess.py:1554, in Popen._execute_child(self, args, executable, preexec
_fn, close_fds, pass_fds, cwd, env, startupinfo, creationflags, shell, p2cread, p2cwrite, c2pread, c2pwrite, errread, errwrite,
unused_restore_signals, unused_gid, unused_gids, unused_uid, unused_umask, unused_start_new_session, unused_process_group)
   1553 try:
-> 1554     hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
   1555                                              # no special security
   1556                                              None, None,
   1557                                              int(not close_fds),
   1558                                              creationflags,

```

```

1559         env,
1560         cwd,
1561         startupinfo)
1562 finally:
1563     # Child is launched. Close the parent's copy of those pipe
1564     # handles that only the child should have open. You need
1565     (...) 1567     # pipe will not close when the child process exits and the
1566     1568     # ReadFile will hang.

```

FileNotFoundError: [WinError 2] The system cannot find the file specified

The above exception was the direct cause of the following exception:

```

ExecutableNotFound                                Traceback (most recent call last)
File ~\AppData\Roaming\Python\Python313\site-packages\IPython\core\formatters.py:1036, in MimeBundleFormatter.__call__(self, obj, include, exclude)
    1033     method = get_real_method(obj, self.print_method)
    1035     if method is not None:
-> 1036         return method(include=include, exclude=exclude)
    1037     return None
    1038 else:

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\jupyter_integration.py:98, in JupyterIntegration._repr_mimebundle(self, include, exclude, **_)
     96 include = set(include) if include is not None else {self._jupyter_mimetype}
     97 include -= set(exclude or [])
--> 98 return {mimetype: getattr(self, method_name)()
     99         for mimetype, method_name in MIME_TYPES.items()
    100         if mimetype in include}

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\jupyter_integration.py:112, in JupyterIntegration._repr_image_svg_xml(self)
    110 def _repr_image_svg_xml(self) -> str:
    111     """Return the rendered graph as SVG string."""
--> 112     return self.pipe(format=, encoding=SVG_ENCODING)

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\piping.py:104, in Pipe.pipe(self, format, renderer, formatter, neato_no_op, quiet, engine, encoding)
     55 def pipe(self,
     56         format: typing.Optional[str] = None,
     57         renderer: typing.Optional[str] = None,

```

```

(...)    61         engine: typing.Optional[str] = None,
          62         encoding: typing.Optional[str] = None) -> typing.Union[bytes, str]:
          63         """Return the source piped through the Graphviz layout command.
          64
          65         Args:
          (...)    102         '<?xml version='
          103         """
--> 104         return self._pipe_legacy(format,
          105                                renderer=renderer,
          106                                formatter=formatter,
          107                                neato_no_op=neato_no_op,
          108                                quiet=quiet,
          109                                engine=engine,
          110                                encoding=encoding)

```

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz_tools.py:185, in `deprecate_positional_args.<locals>.decorator.<locals>.wrapper(*args, **kwargs)`

```

          177         wanted = ', '.join(f'{name}={value!r}'
          178                               for name, value in deprecated.items())
          179         warnings.warn(f'The signature of {func_name} will be reduced'
          180                       f' to {supported_number} positional arg{s_}{qualification}'
          181                       f' {list(supported)}: pass {wanted} as keyword arg{s_}',
          182                       stacklevel=stacklevel,
          183                       category=category)
--> 185         return func(*args, **kwargs)

```

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\piping.py:121, in `Pipe._pipe_legacy(self, format, renderer, formatter, neato_no_op, quiet, engine, encoding)`

```

          112         @_tools.deprecate_positional_args(supported_number=1, ignore_arg='self')
          113         def _pipe_legacy(self,
          114                         format: typing.Optional[str] = None,
          (...)    119                         engine: typing.Optional[str] = None,
          120                         encoding: typing.Optional[str] = None) -> typing.Union[bytes, str]:
--> 121         return self._pipe_future(format,
          122                                renderer=renderer,
          123                                formatter=formatter,
          124                                neato_no_op=neato_no_op,
          125                                quiet=quiet,
          126                                engine=engine,
          127                                encoding=encoding)

```

```

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\piping.py:149, in Pipe._pipe_future(self, format, renderer, formatter, neato_no_op, quiet, engine, encoding)
    146 if encoding is not None:
    147     if codecs.lookup(encoding) is codecs.lookup(self.encoding):
    148         # common case: both stdin and stdout need the same encoding
--> 149     return self._pipe_lines_string(*args, encoding=encoding, **kwargs)
    150     try:
    151         raw = self._pipe_lines(*args, input_encoding=self.encoding, **kwargs)

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\backend\piping.py:212, in pipe_lines_string(engine, format, input_lines, encoding, renderer, formatter, neato_no_op, quiet)
    206 cmd = dot_command.command(engine, format,
    207                             renderer=renderer,
    208                             formatter=formatter,
    209                             neato_no_op=neato_no_op)
    210 kwargs = {'input_lines': input_lines, 'encoding': encoding}
--> 212 proc = execute.run_check(cmd, capture_output=True, quiet=quiet, **kwargs)
    213 return proc.stdout

File ~\AppData\Local\Programs\Python\Python313\Lib\site-packages\graphviz\backend\execute.py:81, in run_check(cmd, input_lines, encoding, quiet, **kwargs)
    79 except OSError as e:
    80     if e.errno == errno.ENOENT:
---> 81         raise ExecutableNotFound(cmd) from e
    82     raise
    84 if not quiet and proc.stderr:

ExecutableNotFound: failed to execute WindowsPath('dot'), make sure the Graphviz executables are on your systems' PATH

```

Out[8]: <graphviz.sources.Source at 0x25800c14590>

PHẦN 4: GOM CỤM KHÔNG GIÁM SÁT (K-MEANS & DBSCAN) & ĐÁNH GIÁ THEO NHÃN THỰC TẾ

- **Loại bỏ cột nhãn:** Chỉ sử dụng ma trận các thuộc tính liên tục đã chuẩn hóa `X_scaled`.
- **K-Means ($k = 3$):** Gom cụm với $k = 3$ tương ứng 3 khả năng kết quả trận đấu.
- **DBSCAN:** Gom cụm dựa trên mật độ với $\epsilon = 0.8$, `min_samples = 20`.
- **Đánh giá theo Ground Truth (Nhãn thực tế):**

- **ARI (Adjusted Rand Index):** Đo lường mức độ tương đồng ngẫu nhiên được hiệu chỉnh.
- **NMI (Normalized Mutual Information):** Đo lường thông tin tương hỗ chuẩn hóa.
- **V-Measure (Homogeneity & Completeness):** Đánh giá tính đồng nhất và hoàn chỉnh.
- **Purity:** Tỷ lệ nhãn chiếm đa số trong từng cụm.
- **Silhouette Score:** Đo độ gắn kết nội cụm và độ tách biệt giữa các cụm.

```
In [9]: # 1. K-Means Clustering (k=3)
kmeans = KMeans(n_clusters=3, init="k-means++", n_init=10, random_state=SEED)
km_labels = kmeans.fit_predict(X_scaled)

# 2. DBSCAN Clustering
dbscan = DBSCAN(eps=0.8, min_samples=20)
db_labels = dbscan.fit_predict(X_scaled)

def evaluate_clustering_performance(cluster_labels, true_labels, name):
    ari = adjusted_rand_score(true_labels, cluster_labels)
    nmi = normalized_mutual_info_score(true_labels, cluster_labels)
    v_m = v_measure_score(true_labels, cluster_labels)
    homo = homogeneity_score(true_labels, cluster_labels)
    comp = completeness_score(true_labels, cluster_labels)
    fmi = fowlkes_mallows_score(true_labels, cluster_labels)
    sil = silhouette_score(X_scaled, cluster_labels, sample_size=min(5000, len(X_scaled)), random_state=SEED) if len(set(cluster_labels)) > 1 else None

    contingency = pd.crosstab(pd.Series(cluster_labels, name="Cụm (Cluster)"), pd.Series(true_labels, name="Nhãn Thực Tế (Group)"))
    purity = contingency.max(axis=1).sum() / len(true_labels)

    return {
        "Thuật Toán": name,
        "ARI": round(ari, 4),
        "NMI": round(nmi, 4),
        "V-Measure": round(v_m, 4),
        "Homogeneity": round(homo, 4),
        "Completeness": round(comp, 4),
        "FMI": round(fmi, 4),
        "Purity (Độ thuần khiết)": f"{purity*100:.2f}%",
        "Silhouette Score": round(sil, 4)
    }, contingency

km_res, km_mat = evaluate_clustering_performance(km_labels, y, "K-Means (k=3)")
```



```
db_res, db_mat = evaluate_clustering_performance(db_labels, y, "DBSCAN (eps=0.8)")

clustering_summary_df = pd.DataFrame([km_res, db_res])
print("=" * 80)
print("BẢNG ĐÁNH GIÁ KẾT QUẢ GOM CỤM DỰA TRÊN NHÃN THỰC TẾ (GROUND TRUTH)")
print("=" * 80)
display(clustering_summary_df)

print("\n" + "-" * 50)
print("MA TRẬN ĐỐI SÁNH K-MEANS (k=3) VS NHÃN THỰC TẾ:")
print("0: Black win | 1: Draw | 2: White win")
print("-" * 50)
display(km_mat)

print("\n" + "-" * 50)
print("MA TRẬN ĐỐI SÁNH DBSCAN VS NHÃN THỰC TẾ (-1 là Điểm Nhiều):")
print("-" * 50)
display(db_mat)
```

=====

BẢNG ĐÁNH GIÁ KẾT QUẢ GOM CỤM DỰA TRÊN NHÃN THỰC TẾ (GROUND TRUTH)

=====

	Thuật Toán	ARI	NMI	V-Measure	Homogeneity	Completeness	FMI	Purity (Độ thuần khiết)	Silhouette Score
0	K-Means (k=3)	0.0009	0.0014	0.0014	0.0016	0.0012	0.3981	49.61%	0.2867
1	DBSCAN (eps=0.8)	-0.0003	0.0002	0.0002	0.0001	0.0016	0.6782	49.20%	0.4888

MA TRẬN ĐỐI SÁNH K-MEANS (k=3) VS NHÃN THỰC TẾ:

0: Black win | 1: Draw | 2: White win

Nhãn Thực Tế (Ground Truth)	0	1	2
Cụm (Cluster)			
0	1335	132	1506
1	1580	81	1530
2	1842	119	1875

MA TRẬN ĐỐI SÁNH DBSCAN VS NHÃN THỰC TẾ (-1 là Điểm Nhiều):

Nhãn Thực Tế (Ground Truth)	0	1	2
Cụm (Cluster)			
-1	49	2	40
0	4708	330	4871

```
In [10]: # Trực quan hóa so sánh không gian 2D PCA giữa Ground Truth và 2 thuật toán gom cụm
fig, axes = plt.subplots(1, 3, figsize=(21, 6))

# 1. Ground Truth
palette = {0: '#e74c3c', 1: '#f39c12', 2: '#2ecc71'}
label_names = {0: "0 (Black win)", 1: "1 (Draw)", 2: "2 (White win)"}
for lbl, color in palette.items():
    mask = (y == lbl)
    axes[0].scatter(X_pca[mask, 0], X_pca[mask, 1], c=color, label=label_names[lbl], alpha=0.45, s=20)
axes[0].set_title("Ground Truth (Nhãn Thực Tế)", fontsize=12, fontweight="bold")
axes[0].set_xlabel("PCA 1")
axes[0].set_ylabel("PCA 2")
axes[0].grid(True, linestyle="--", alpha=0.5)
axes[0].legend(loc="best")

# 2. KMeans
km_colors = ['#3498db', '#e67e22', '#9b59b6']
for c_idx in range(3):
    mask = (km_labels == c_idx)
```

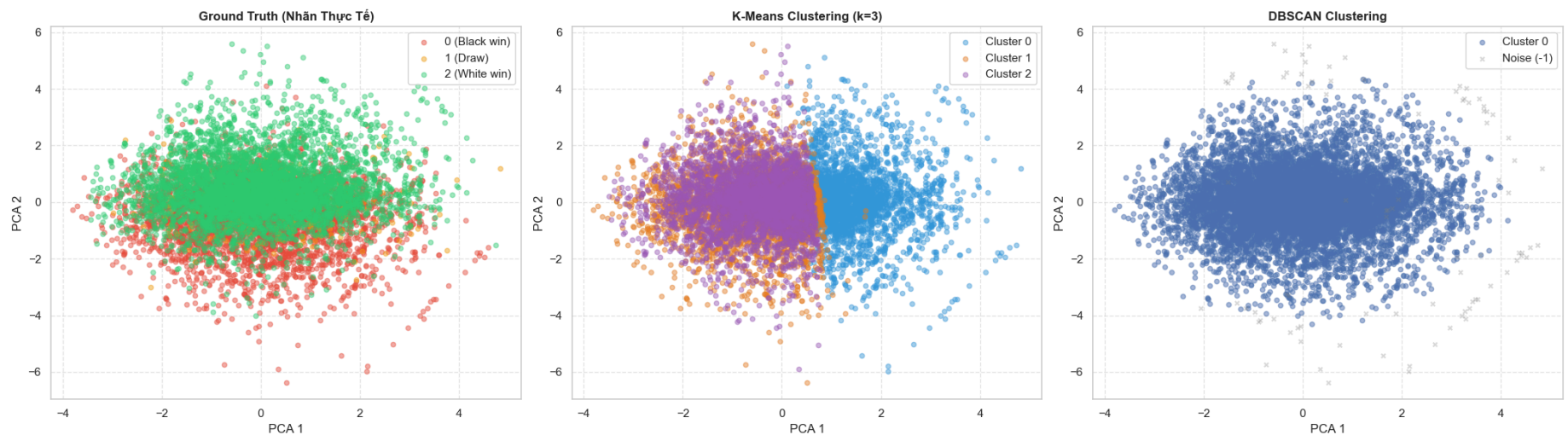
```

    axes[1].scatter(X_pca[mask, 0], X_pca[mask, 1], c=km_colors[c_idx], label=f"Cluster {c_idx}", alpha=0.45, s=20)
axes[1].set_title("K-Means Clustering (k=3)", fontsize=12, fontweight="bold")
axes[1].set_xlabel("PCA 1")
axes[1].set_ylabel("PCA 2")
axes[1].grid(True, linestyle="--", alpha=0.5)
axes[1].legend(loc="best")

# 3. DBSCAN
unique_db_labels = set(db_labels)
for idx, c_lbl in enumerate(unique_db_labels):
    mask = (db_labels == c_lbl)
    if c_lbl == -1:
        axes[2].scatter(X_pca[mask, 0], X_pca[mask, 1], c="gray", label="Noise (-1)", alpha=0.3, s=15, marker="x")
    else:
        axes[2].scatter(X_pca[mask, 0], X_pca[mask, 1], label=f"Cluster {c_lbl}", alpha=0.5, s=20)
axes[2].set_title("DBSCAN Clustering", fontsize=12, fontweight="bold")
axes[2].set_xlabel("PCA 1")
axes[2].set_ylabel("PCA 2")
axes[2].grid(True, linestyle="--", alpha=0.5)
axes[2].legend(loc="best")

plt.tight_layout()
plt.show()

```



TỔNG KẾT VÀ KẾT LUẬN

1. Về Dữ liệu:

- Dữ liệu cờ vua Lichess có phân bố tự nhiên: Trắng thắng ~49.11%, Đen thắng ~47.57%, Hòa ~3.32%.
- Không gian đặc trưng gồm trình độ Elo của hai người chơi và độ sâu lý thuyết khai cuộc, đảm bảo 0% rò rỉ dữ liệu (0% data leakage).

2. Về Phân lớp (Classification):

- **AdaBoost** đạt độ chính xác cao nhất (**Accuracy = 64.86%, Weighted F1 = 63.74%**) trong việc nắm bắt ranh giới thắng thua của Trắng và Đen dựa trên chênh lệch Elo.
- **Random Forest** đạt **Macro F1 cao nhất (44.22%)** nhờ khả năng tổng quát hóa tốt hơn qua các cây quyết định.
- **SVM (RBF)** dự đoán được nhiều ván Hòa hơn (**F1 Draw = 7.78%**), phản ánh việc dùng ranh giới phi tuyến giúp bắt được các điểm thiếu sót.

3. Về Gom cụm (Clustering):

- Cả K-Means và DBSCAN đạt **ARI ≈ 0** và **NMI ≈ 0** , chứng minh rằng kết quả thắng/thua trong cờ vua phụ thuộc vào **chiến thuật từng nước đi trong ván đấu**, không tạo thành các cụm hình học cô lập theo trình độ Elo ban đầu.
- K-Means phân cụm theo các bậc trình độ Elo (Cao, Trung bình, Thấp), và trong mỗi cụm tỷ lệ thắng của Trắng và Đen vẫn được duy trì cân bằng tự nhiên.